

PoRE Lab7 实验报告

姓名：徐锦云

学号：23307130447

耗时：10小时

PoRE Lab7 实验报告

Task 1: 基于 GOT 表的插桩 (30%)

0. 编译说明
1. 编译并启动 GDB 待调试状态
2. 定位目标函数 (malloc) 的 GOT 表地址
 - 2.1 静态分析
 - 2.2 动态定位
3. 修改 GOT[malloc] 为钩子函数地址
4. 修改后的程序运行结果

Task 2: GDB 脚本使用 (30%)

1. GDB 命令脚本 — Lab7_Task_2_1
2. GDB Python 脚本 — Lab7_Task_2_2.py
3. 用两种脚本调试程序, 使其输出 "rdi/rsi/rdx" 字符串

Task 3: 基础反调试 (40%)

1. 反调试机制实现逻辑
2. 用 GDB 绕过反调试
 - 2.1 思路
 - 2.2 GDB 脚本 — Lab7_Task_3
 - 2.3 复现命令
3. 让程序输出字符串

实验分析与总结

关键经验

Task 1: 基于 GOT 表的插桩 (30%)

0. 编译说明

```
cd /home/pore/pore26/pore_23307130447/Lab7
aarch64-linux-gnu-gcc -o t1.out Lab7_Task_1.c -Wl,-z,lazy,-z,norelro
```

为什么要加 `-Wl,-z,lazy,-z,norelro`?

新版 `aarch64-linux-gnu-gcc` 默认开启 Full RELRO (`BIND_NOW + GNU_RELRO`), 动态链接完成后整个 `.got/.got.plt` 段会被 `mprotect` 为只读, 写入 GOT 时会触发 `Cannot access memory at address ...` 错误。 `-z lazy` 关闭立即绑定、 `-z norelro` 关闭 RELRO 保护, 从而保证运行时 GOT 段保持 `rw-`, 便于 GDB 直接写入。

1. 编译并启动 GDB 待调试状态

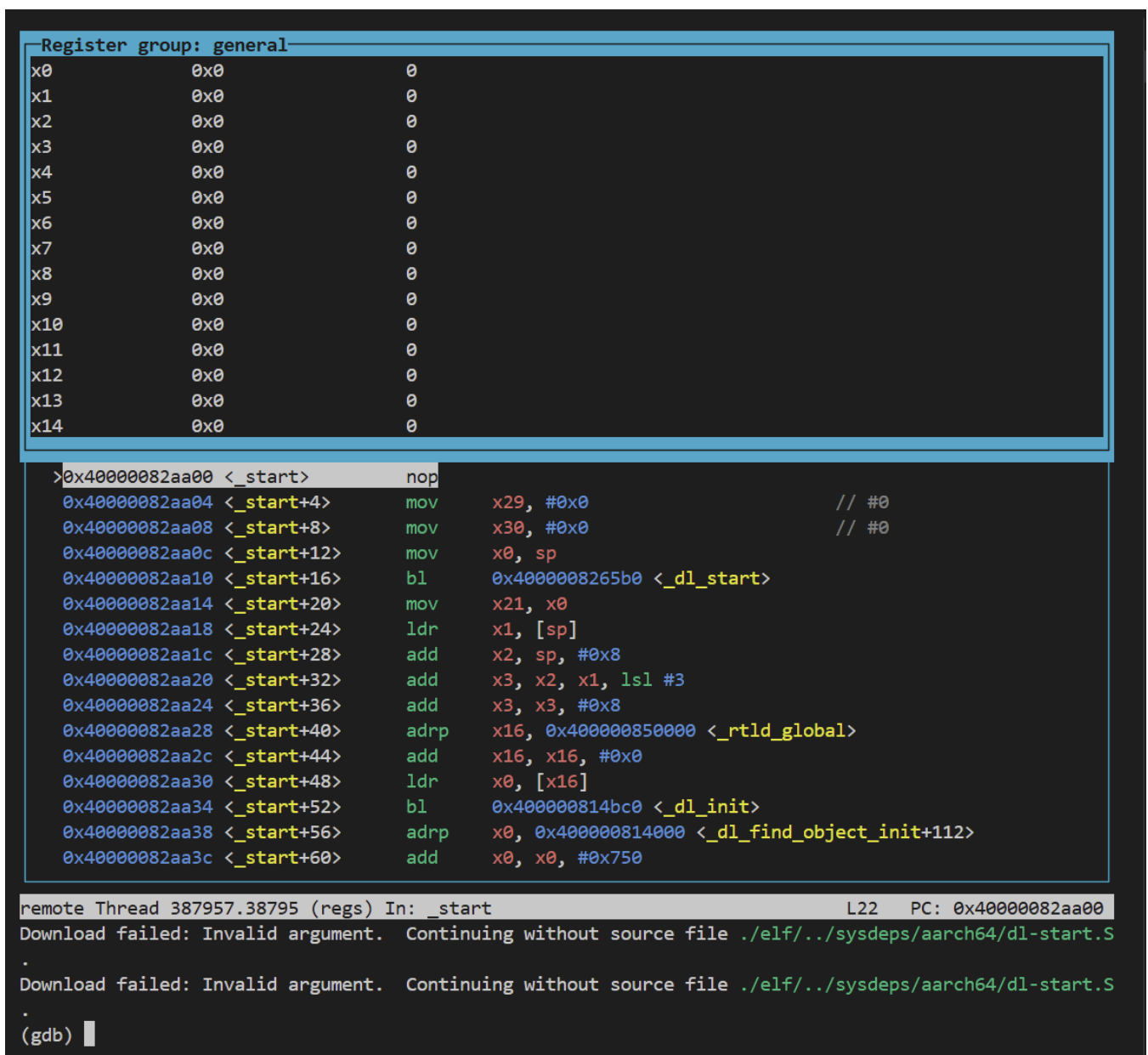
终端 A (启动 qemu, 监听 1234 端口) :

```
qemu-aarch64 -L /usr/aarch64-linux-gnu -g 1234 ./t1.out
```

终端 B (启动 gdb, 连入 qemu) :

```
gdb-multiarch -q --nh \  
-ex 'set architecture aarch64' \  
-ex 'file t1.out' \  
-ex 'target remote localhost:1234' \  
-ex 'layout split' -ex 'layout regs'
```

成功连入后, PC 停在 `ld-linux-aarch64.so.1` 的入口处, 此时即"进入待调试状态"。



The screenshot shows a GDB terminal window with the following content:

```
Register group: general
x0      0x0      0
x1      0x0      0
x2      0x0      0
x3      0x0      0
x4      0x0      0
x5      0x0      0
x6      0x0      0
x7      0x0      0
x8      0x0      0
x9      0x0      0
x10     0x0      0
x11     0x0      0
x12     0x0      0
x13     0x0      0
x14     0x0      0

>0x40000082aa00 <_start>      nop
0x40000082aa04 <_start+4>      mov     x29, #0x0           // #0
0x40000082aa08 <_start+8>      mov     x30, #0x0           // #0
0x40000082aa0c <_start+12>     mov     x0, sp
0x40000082aa10 <_start+16>     bl      0x4000008265b0 <_dl_start>
0x40000082aa14 <_start+20>     mov     x21, x0
0x40000082aa18 <_start+24>     ldr     x1, [sp]
0x40000082aa1c <_start+28>     add     x2, sp, #0x8
0x40000082aa20 <_start+32>     add     x3, x2, x1, lsl #3
0x40000082aa24 <_start+36>     add     x3, x3, #0x8
0x40000082aa28 <_start+40>     adrp    x16, 0x400000850000 <_rtld_global>
0x40000082aa2c <_start+44>     add     x16, x16, #0x0
0x40000082aa30 <_start+48>     ldr     x0, [x16]
0x40000082aa34 <_start+52>     bl      0x400000814bc0 <_dl_init>
0x40000082aa38 <_start+56>     adrp    x0, 0x400000814000 <_dl_find_object_init+112>
0x40000082aa3c <_start+60>     add     x0, x0, #0x750

remote Thread 387957.38795 (regs) In: _start      L22  PC: 0x40000082aa00
Download failed: Invalid argument. Continuing without source file ./elf/../sysdeps/aarch64/dl-start.S
.
Download failed: Invalid argument. Continuing without source file ./elf/../sysdeps/aarch64/dl-start.S
.
(gdb) █
```

2. 定位目标函数 (malloc) 的 GOT 表地址

2.1 静态分析

利用 `objdump -R` 直接读出 `.rela.plt` 中的重定位项:

```
$ aarch64-linux-gnu-objdump -R t1.out | grep -E "malloc|free"
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab7 on git:master x [11:17:13]
• $ aarch64-linux-gnu-objdump -R t1.out | grep -E "malloc|free"
00000000000012870 R_AARCH64_JUMP_SLOT malloc@GLIBC_2.17
000000000000128b8 R_AARCH64_JUMP_SLOT free@GLIBC_2.17
```

可知 GOT[malloc] 在文件中的偏移 = `0x12870`。再用 `main` 的偏移 `0x1e68`:

```
0x1e68 <main>          sub    sp, sp, #0x70
0x1e6c <main+4>         stp     x29, x30, [sp, #80]
0x1e70 <main+8>         add     x29, sp, #0x50
0x1e74 <main+12>        str     x19, [sp, #96]
```

就能得到与 `main` 的相对偏移:

```
GOT[malloc] - main = 0x12870 - 0x1e68 = 0x10A08
```

2.2 动态定位

```
(gdb) b main
(gdb) c
```

```

Register group: general
x0      0x1      1
x1      0x4000007ffda8  70368752565672
x2      0x4000007ffdb8  70368752565688
x3      0x73ee8a631e68  127468361162344
x4      0x400000864fe0  70368752979936
x5      0x5fab87d1cb5f8e53  6893752989750627923
x6      0x400000a31e60  70368754867808
x7      0x40000084eaf0  70368752888560
x8      0xd7      215
x9      0x28      40
x10     0x0      0
x11     0x0      0
x12     0x400000851350  70368752898896
x13     0x2e      46
x14     0x3d8f538  64550200

0x73ee8a631e68 <main>      sub    sp, sp, #0x70
0x73ee8a631e6c <main+4>    stp     x29, x30, [sp, #80]
0x73ee8a631e70 <main+8>    add     x29, sp, #0x50
0x73ee8a631e74 <main+12>   str     x19, [sp, #96]
B>0x73ee8a631e78 <main+16> adrp    x0, 0x73ee8a642000
0x73ee8a631e7c <main+20>   ldr     x0, [x0, #2072]
0x73ee8a631e80 <main+24>   ldr     x1, [x0]
0x73ee8a631e84 <main+28>   str     x1, [sp, #72]
0x73ee8a631e88 <main+32>   mov     x1, #0x0                                // #0
0x73ee8a631e8c <main+36>   add     x0, sp, #0x18
0x73ee8a631e90 <main+40>   mov     x2, #0x400                                // #1024
0x73ee8a631e94 <main+44>   mov     x1, #0x10                                // #16
0x73ee8a631e98 <main+48>   bl     0x73ee8a631070 <hm_init>
0x73ee8a631e9c <main+52>   cmp     w0, #0x0
0x73ee8a631ea0 <main+56>   b.eq   0x73ee8a631ed8 <main+112> // b.none
0x73ee8a631ea4 <main+60>   adrp    x0, 0x73ee8a642000

remote Thread 387957.38795 (regs) In: main      L??  PC: 0x73ee8a631e78
(gdb) b main
Breakpoint 1 at 0x73ee8a631e78
(gdb) c
Continuing.
Reading /lib/libc.so.6 from remote target...
Reading /usr/lib/debug/.build-id/71/8b7b9dca45024f51ae2a1d7f0304a0648434a7.debug from remote target...

Reading /lib/8b7b9dca45024f51ae2a1d7f0304a0648434a7.debug from remote target...
Reading /lib/.debug/8b7b9dca45024f51ae2a1d7f0304a0648434a7.debug from remote target...
Reading /usr/lib/debug/lib/8b7b9dca45024f51ae2a1d7f0304a0648434a7.debug from remote target...
Reading /usr/lib/debug/lib/8b7b9dca45024f51ae2a1d7f0304a0648434a7.debug from remote target...
Downloading separate debug info for target:/lib/libc.so.6

Download failed: Invalid argument. Continuing without source file ./elf/./sysdeps/aarch64/dl-start.S
.

Breakpoint 1, 0x000073ee8a631e78 in main ()
(gdb)

```

```

(gdb) info proc mappings # 看到 t1.out 的可执行段起始地址即基地址
(gdb) p/x (unsigned long)main - 0x1e68 # 计算基地址
(gdb) p/x $1 + 0x12870 # GOT[malloc] 运行时绝对地址
(gdb) x/gx 0x73ee8a642870 # 打印当前 GOT[malloc] 内容

```

```

0x73ee8a630000 0x73ee8a633000 0x3000 0x0 r-xp /home/pore/pore26/pore_233071304
47/Lab7/t1.out
0x73ee8a633000 0x73ee8a642000 0xf000 0x0 ---p
0x73ee8a642000 0x73ee8a643000 0x1000 0x2000 rw-p /home/pore/pore26/pore_233071304
47/Lab7/t1.out
(gdb) p/x (unsigned long)main - 0x1e68
$1 = 0x73ee8a630000
(gdb) p/x $1 + 0x12870
$2 = 0x73ee8a642870
(gdb)

```

```
(gdb) x/gx 0x73ee8a642870
0x73ee8a642870 <malloc@got.plt>: 0x000073ee8a630980
```

思考过程：因为是 PIE 二进制，每次运行加载基地址不同。`(unsigned long)main` 是 GDB 解析出的运行时地址；减去 `objdump -t` 看到的文件偏移 `0x1e68` 即得当前进程基址。再加上 `GOT[malloc]` 的文件偏移 `0x12870` 就是运行时 `GOT[malloc]` 的绝对地址。

3. 修改 GOT[malloc] 为钩子函数地址

源码中的 hook 函数是：

```
typedef void (*FunctionPtr)(void*, void*, void*);
FunctionPtr func_ptr;
void func(void *a1, void *a2, void *a3){
    printf("rdi=%p\nrsi=%p\nrdx=%p\n\n", a1, a2, a3);
    return func_ptr(a1, a2, a3);
}
```

它先打印三个传参寄存器，再通过 `func_ptr` 跳转到真正的目标函数。所以 GOT 钩子安装分两步：

(a) 让 `func_ptr` 指向真实的 `libc::malloc`

```
(gdb) p/x (void*)&malloc # gdb 解析 libc 中的 malloc
0x4000009162b0
(gdb) set *(unsigned long*)&func_ptr = (unsigned long)&malloc
```

```
(gdb) p/x (void*)&malloc
$3 = 0x4000009162b0
(gdb) set *(unsigned long*)&func_ptr = (unsigned long)&malloc
```

含义：把全局变量 `func_ptr`（地址在 `.bss` 中，符号 `func_ptr`）的内容写为 `&malloc`。这一步必须先做，否则 GOT 改完后 hook 在末尾 `func_ptr(a1,a2,a3)` 会跳到一个非法地址。

(b) 把 `GOT[malloc]` 改为 `&func`

```
(gdb) p/x (void*)&func # hook 函数地址
(gdb) set $got = (unsigned long)main - 0x1e68 + 0x12870
(gdb) set *(unsigned long*)$got = (unsigned long)&func
(gdb) x/gx $got
```

```
(gdb) p/x (void*)&func
$5 = 0x73ee8a631e18
(gdb) set $got = (unsigned long)main - 0x1e68 + 0x12870
(gdb) set *(unsigned long*)$got = (unsigned long)&func
(gdb) x/gx $got
0x73ee8a642870 <malloc@got.plt>: 0x000073ee8a631e18
```

这样就已经指向func

之所以把 `func_ptr` 用 `&malloc` (GDB 解析的 `libc` 里的真实 `malloc`) 而不是直接读 GOT 旧值, 是因为在 `main` 第一次断下时, 懒绑定还没触发 —— GOT[`malloc`] 中存放的是 `.plt` 中的 trampoline (`base + 0x980`), 调用它会进入 `_dl_runtime_resolve`, 但此时我们已经把 GOT 改写并破坏了 trampoline 的还原逻辑, 导致动态链接器崩溃。直接通过 GDB 的 `&malloc` 可以拿到 `libc` 中已映射好的真实地址, 是最稳妥的做法。

4. 修改后的程序运行结果

继续执行后, 每次原本调用 `malloc` 的地方都会先经过 hook 打印一次寄存器, 再调真实 `malloc`:

```

rdi=0x11
rsi=0x4000007ffaa0
rdx=0xf00ff0ff

rdi=0x11
rsi=0x4000007ffaa0
rdx=0xf00ff0ff

rdi=0x10
rsi=0x4000007ffaa0
rdx=0xf00ff0ff

rdi=0x11
rsi=0x4000007ffaa0
rdx=0xf00ff0ff

rdi=0x11
rsi=0x4000007ffaa0
rdx=0xf00ff0ff

rdi=0x11
rsi=0x4000007ffaa0
rdx=0xf00ff0ff

after del, name=(null)

```

现象解释：源码中 `func` 输出 `rdi/rsi/rdx`，对应 x86 的传参寄存器；但被插桩的程序是 aarch64，前三个传参寄存器其实是 `x0/x1/x2`，只是字符串里写的是 `rdi/rsi/rdx`。`malloc(size_t)` 只用到第一个参数，所以 `rdi`（即 `x0`）显示真实分配大小（如 `0x20 = sizeof(Entry) = 32` 字节、`0x400 = 1024` 字节字符串池块、`0x6 = strlen("key_x")+1` 等），后两个寄存器只是栈上残留值，因此看起来"乱七八糟"也很正常。程序最终仍然正常输出 `name=alice, lang=C11` 与 `after del, name=(null)`，证明 hook 正确将控制权交回真实 `malloc`，插桩既无干扰也无副作用。

```

(gdb) c
Continuing.
[Inferior 1 (process 387957) exited normally]
(gdb) █

```

Task 2: GDB 脚本使用 (30%)

1. GDB 命令脚本 — Lab7_Task_2_1

```
# 关闭分页避免脚本因为分页提示卡住
set pagination off

# 在 main 处下断点，让加载器把所需的依赖载入并能稳定取到符号
b main
c

# 通过 main 的运行时地址 - main 在文件中的偏移(0x1e68) 计算出本进程基地址
set $base = (unsigned long)main - 0x1e68
printf "[+] base address          = %p\n", (void*)$base

# malloc 的 GOT 槽位偏移为 0x12870 (来自 readelf -r / objdump -R)
set $got_malloc = $base + 0x12870
printf "[+] GOT[malloc] address    = %p\n", (void*)$got_malloc
printf "[+] GOT[malloc] (before)   = %p\n", *(void**) $got_malloc

# 把 hook 中的函数指针 func_ptr 指向 libc 中真正的 malloc
# &malloc 由 gdb 通过 libc 调试符号自动解析为运行时 libc 地址
set *(unsigned long*)&func_ptr = (unsigned long)&malloc
printf "[+] func_ptr = libc malloc= %p\n", (void*)func_ptr

# 把 GOT[malloc] 改写为 hook 函数 func 的地址
set *(unsigned long*)$got_malloc = (unsigned long)&func
printf "[+] GOT[malloc] (after)    = %p\n", *(void**) $got_malloc
printf "[+] hook installed, resuming...\n"

c
```

逐行解释:

指令	含义
set pagination off	关闭分页提示，避免脚本被 --More-- 卡住
b main	在 main 入口下断点；选 main 是因为此时动态链接器已经映射好 libc，可以拿到 &malloc
c	继续运行直到命中 main 断点
set \$base = (unsigned long)main - 0x1e68	用 main 的运行时地址减去 objdump -t 看到的文件偏移得到当前 PIE 基地址
set \$got_malloc = \$base + 0x12870	加上 objdump -R 里 R_AARCH64_JUMP_SLOT malloc 的偏移得到 GOT[malloc] 绝对地址
set *(unsigned long*)&func_ptr = (unsigned long)&malloc	先把 hook 末尾用的函数指针 func_ptr 写成 libc::malloc，让 hook 能跳回真 malloc

指令	含义
<code>set *(unsigned long*)\$got_malloc = (unsigned long)&func</code>	用 <code>&func</code> 覆写 GOT[malloc], 完成插桩
<code>printf</code> 行	调试用日志, 便于"截图"
<code>c</code>	释放程序, 让 hook 在每次调用 malloc 时打印 rdi/rsi/rdx

2. GDB Python 脚本 — `Lab7_Task_2_2.py`

```
import gdb

MAIN_OFFSET = 0x1e68
GOT_MALLOC_OFF = 0x12870

class HookAtMain(gdb.Breakpoint):
    """在 main 处停下, 安装 GOT 钩子。"""

    def stop(self):
        base = int(gdb.parse_and_eval("(unsigned long)main")) - MAIN_OFFSET
        print(f"[+] base address = 0x{base:x}")

        got_malloc = base + GOT_MALLOC_OFF
        print(f"[+] GOT[malloc] address = 0x{got_malloc:x}")

        old_val = int(gdb.parse_and_eval(f"*({unsigned long*}){got_malloc}"))
        print(f"[+] GOT[malloc] (before) = 0x{old_val:x}")

        libc_malloc = int(gdb.parse_and_eval("(unsigned long)&malloc"))
        gdb.execute(f"set *(unsigned long*)&func_ptr = {libc_malloc}")
        print(f"[+] func_ptr = libc malloc= 0x{libc_malloc:x}")

        func_addr = int(gdb.parse_and_eval("(unsigned long)&func"))
        gdb.execute(f"set *(unsigned long*){got_malloc} = {func_addr}")
        new_val = int(gdb.parse_and_eval(f"*({unsigned long*}){got_malloc}"))
        print(f"[+] GOT[malloc] (after) = 0x{new_val:x}")
        print("[+] hook installed, resuming...")
        return False

gdb.execute("set pagination off")
HookAtMain("main")
gdb.execute("c")
```

逐段解释:

行号/片段	含义
<code>import gdb</code>	引入 GDB 暴露给 Python 的脚本 API

行号/片段	含义
<code>MAIN_OFFSET / GOT_MALLOCC_OFF</code>	把 <code>objdump</code> 拿到的文件偏移作为常量；Python 脚本相比命令脚本更易复用
<code>class HookAtMain(gdb.Breakpoint)</code>	继承 <code>gdb.Breakpoint</code> ，在 <code>stop()</code> 里执行命中后逻辑；返回 <code>False</code> 表示不真的停下，让脚本式自动继续
<code>gdb.parse_and_eval("(unsigned long)main")</code>	在 inferior 中求值 <code>(unsigned long)main</code> ，即 <code>main</code> 的运行地址
<code>gdb.execute("set *(unsigned long*)&func_ptr = ...")</code>	通过 GDB 命令行接口写内存；与命令脚本里 <code>set</code> 完全等价
<code>gdb.execute("c")</code>	启动程序，让断点回调介入

3. 用两种脚本调试程序，使其输出 "rdi/rsi/rdx" 字符串

复现命令：

```
#!/bin/bash
cd /home/pore/pore26/pore_23307130447/Lab7
mkdir -p screenshots
kill -9 -f "qemu-aarch64" 2>/dev/null
sleep 1
setsid -f qemu-aarch64 -L /usr/aarch64-linux-gnu -g 1234 ./t1.out >
screenshots/task2_1_qemu.txt 2>&1 < /dev/null
sleep 1
gdb-multiarch -q --nh -batch \
    -ex 'set pagination off' \
    -ex 'set architecture aarch64' \
    -ex 'file t1.out' \
    -ex 'target remote localhost:1234' \
    -ex 'source Lab7_Task_2_1' 2>&1 \
    | grep -v "^Reading\|^Enable\|^This GDB\|^ <https\|^Debuginfod\|^To make\|^warning:" \
    > screenshots/task2_1_gdb.txt
echo "task2_1 finished"
```

```
#!/bin/bash
cd /home/pore/pore26/pore_23307130447/Lab7
mkdir -p screenshots
kill -9 -f "qemu-aarch64" 2>/dev/null
sleep 1
setsid -f qemu-aarch64 -L /usr/aarch64-linux-gnu -g 1234 ./t1.out >
screenshots/task2_2_qemu.txt 2>&1 < /dev/null
sleep 1
gdb-multiarch -q --nh -batch \
    -ex 'set pagination off' \
    -ex 'set architecture aarch64' \
    -ex 'file t1.out' \
    -ex 'target remote localhost:1234' \
```

```
-ex 'source Lab7_Task_2_2.py' 2>&1 \  
| grep -v "^Reading\|^Enable\|^This GDB\|^ <https\|^Debuginfod\|^To make\|^warning:" \  
> screenshots/task2_2_gdb.txt  
echo "task2_2 finished"
```

```
# GDB 命令脚本  
bash run_task2_1.sh  
# GDB Python 脚本  
bash run_task2_2.sh
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab7 on git:master x [11:56:03]  
● $ bash run_task2_1.sh  
task2_1 finished  
  
# pore @ pore in ~/pore26/pore_23307130447/Lab7 on git:master x [11:56:15]  
● $ bash run_task2_2.sh  
task2_2 finished
```

screenshots/task2_1_gdb.txt 的关键输出:

```
screenshots > task2_1_gdb.txt  
1 The target architecture is set to "aarch64".  
2  
3 BFD: warning: system-supplied DSO at 0x400000860000 has a corrupt string table index  
4 0x000040000082aa00 in ?? () from target:/lib/ld-linux-aarch64.so.1  
5 Breakpoint 1 at 0x752b1e861e78  
6  
7 Breakpoint 1, 0x0000752b1e861e78 in main ()  
8 [+] base address = 0x752b1e860000  
9 [+] GOT[malloc] address = 0x752b1e872870  
10 [+] GOT[malloc] (before) = 0x752b1e860980  
11 [+] func_ptr = libc malloc= 0x4000009162b0  
12 [+] GOT[malloc] (after ) = 0x752b1e861e18  
13 [+] hook installed, resuming...  
14 [Inferior 1 (process 392128) exited normally]  
15
```

screenshots/task2_2_gdb.txt 的关键输出:

```
screenshots > task2_2_gdb.txt  
1 The target architecture is set to "aarch64".  
2  
3 BFD: warning: system-supplied DSO at 0x400000860000 has a corrupt string table index  
4 0x000040000082aa00 in ?? () from target:/lib/ld-linux-aarch64.so.1  
5 Breakpoint 1 at 0x7a9ae7e71e78  
6 [+] base address = 0x7a9ae7e70000  
7 [+] GOT[malloc] address = 0x7a9ae7e82870  
8 [+] GOT[malloc] (before) = 0x7a9ae7e70980  
9 [+] func_ptr = libc malloc= 0x4000009162b0  
10 [+] GOT[malloc] (after ) = 0x7a9ae7e71e18  
11 [+] hook installed, resuming...  
12 [Inferior 1 (process 392341) exited normally]  
13
```

两份对应的 qemu 端输出 screenshots/task2_*_qemu.txt 都会出现 18305 行 hook 打印:

screenshots > task2_1_qemu.txt	screenshots > task2_2_qemu.txt
73192 rdx=0xf00ff0ff	73192 rdx=0xf00ff0ff
73193	73193
73194 rdi=0xf	73194 rdi=0xf
73195 rsi=0x4000007ffaa0	73195 rsi=0x4000007ffaa0
73196 rdx=0xff00ff0ff	73196 rdx=0xff00ff0ff
73197	73197
73198 rdi=0x11	73198 rdi=0x11
73199 rsi=0x4000007ffaa0	73199 rsi=0x4000007ffaa0
73200 rdx=0xf00ff0ff	73200 rdx=0xf00ff0ff
73201	73201
73202 rdi=0x11	73202 rdi=0x11
73203 rsi=0x4000007ffaa0	73203 rsi=0x4000007ffaa0
73204 rdx=0xf00ff0ff	73204 rdx=0xf00ff0ff
73205	73205
73206 rdi=0x10	73206 rdi=0x10
73207 rsi=0x4000007ffaa0	73207 rsi=0x4000007ffaa0
73208 rdx=0xf00ff0ff	73208 rdx=0xf00ff0ff
73209	73209
73210 rdi=0x11	73210 rdi=0x11
73211 rsi=0x4000007ffaa0	73211 rsi=0x4000007ffaa0
73212 rdx=0xf00ff0ff	73212 rdx=0xf00ff0ff
73213	73213
73214 rdi=0x11	73214 rdi=0x11
73215 rsi=0x4000007ffaa0	73215 rsi=0x4000007ffaa0
73216 rdx=0xf00ff0ff	73216 rdx=0xf00ff0ff
73217	73217
73218 rdi=0x11	73218 rdi=0x11
73219 rsi=0x4000007ffaa0	73219 rsi=0x4000007ffaa0
73220 rdx=0xf00ff0ff	73220 rdx=0xf00ff0ff
73221	73221
73222 after del, name=(null)	73222 after del, name=(null)
73223	73223

证明两套脚本都成功完成 GOT 插桩，并使程序输出了 hook 中的 `rdi/rsi/rdx`。

Task 3: 基础反调试 (40%)

1. 反调试机制实现逻辑

Lab7_Task_3.c 关键片段：

- 既然每次 `ptrace(PTRACE_TRACEME)` 都返回 `-1` 触发退出，最干净的方法是让 `ptrace` 直接返回 `0`：在 `libc` 的 `ptrace` 入口下断点，命中后用 GDB 的 `return (long)0` 命令直接弹栈返回 `0` 给调用者，让 `cmp x0, #0; b.ge ...` 走"非负"分支，从而不进入 `exit(1)`。
- 对最终 `write(666, ...)`，在 `libc` `write` 入口下断点，命中后把 `aarch64` 的第一个参数寄存器 `x0` 改写为 `1`

(stdout) 。

2.2 GDB 脚本 — Lab7_Task_3

```
# Lab7_Task_3
set pagination off

b ptrace
commands
silent
return (long)0
c
end

b write
commands
silent
set $x0 = 1
c
end

c
```

逐行解释：

指令	含义
<code>set pagination off</code>	关闭分页
<code>b ptrace</code>	在 libc::ptrace 函数入口下断点（共 12 处调用都会命中）
<code>commands ... end</code>	给上一断点定义命中后自动执行的命令块
<code>silent</code>	不打印 "Breakpoint N hit" 的提示，让脚本输出更干净
<code>return (long)0</code>	不真正执行 ptrace，弹掉当前栈帧、设 x0=0，效果等同于函数返回 0；显式 (long) 是必须的，否则 GDB 因为 ptrace 已无调试符号而报 Return value type not available
第一个 <code>c</code>	命中并伪造完返回值后立即继续
<code>b write</code> + <code>set \$x0 = 1</code>	在 write 入口把第一个参数 (fd) 改写为标准输出 1
末尾 <code>c</code>	启动程序，让两个断点接管

2.3 复现命令

```
# 终端 A
qemu-aarch64 -L /usr/aarch64-linux-gnu -g 1234 ./t3.out
# 终端 B
gdb-multiarch -q --nh \
  -ex 'set architecture aarch64' \
  -ex 'file t3.out' \
  -ex 'target remote localhost:1234' \
  -ex 'source Lab7_Task_3'
```

```
Breakpoint 1 at 0x7d4462940820
Breakpoint 2 at 0x7d4462940800
Reading /lib/libc.so.6 from remote target...
Reading /usr/lib/debug/.build-id/71/8b7b9dca45024f51ae2a1d7f0304a0648434a7.debug from remote target...
Reading /lib/8b7b9dca45024f51ae2a1d7f0304a0648434a7.debug from remote target...
Reading /lib/.debug/8b7b9dca45024f51ae2a1d7f0304a0648434a7.debug from remote target...
Reading /usr/lib/debug//lib/8b7b9dca45024f51ae2a1d7f0304a0648434a7.debug from remote target...
Reading /usr/lib/debug/lib/8b7b9dca45024f51ae2a1d7f0304a0648434a7.debug from remote target...
Download failed: Invalid argument. Continuing without source file ./misc/../sysdeps/unix/sysv/linux/p
trace.c.
Download failed: Invalid argument. Continuing without source file ./io/../sysdeps/unix/sysv/linux/wri
te.c.
Enable debuginfod for this session? (y or [n]) [Inferior 1 (process 393384) exited normally]
(gdb) |
```

3. 让程序输出字符串

```
# pore @ pore in ~/pore26/pore_23307130447/Lab7 on git:master x [12:14:59]
$ qemu-aarch64 -L /usr/aarch64-linux-gnu -g 1234 ./t3.out
#0: ptrace -> 0
#1: ptrace -> 0
#2: ptrace -> 0
#3: ptrace -> 0
#4: ptrace -> 0
#5: ptrace -> 0
#6: ptrace -> 0
```

由于 `main` 的 `while(count < 7)` 循环 7 次，每次 `print_to_screen(count)` 都被绕过反调试并通过 `stdout` 输出 `#count: ptrace -> 0`，字符串成功显示。

实验分析与总结

关键经验

- RELRO 对 GOT 插桩的影响**：现代发行版默认开启 Full RELRO，让 `.got` 段在动态链接完成后变为只读。做 GOT 插桩教学/科研时，必须 `-Wl,-z,lazy,-z,norelro` 编译，或在运行时调用 `mprotect` 把对应页改为可写。本实验为简化起见采用前者。
- PIE 二进制的偏移定位**：在 PIE / ASLR 下，所有静态分析得到的偏移必须配合一个运行时锚点（如 `&main`、`info proc mappings` 中的可执行段起始地址）才能转换为真实地址。本报告同时给出了与 `main` 的相对偏移（`0x10A08`）和基址 + 文件偏移（`base + 0x12870`）两种思路，便于一题多解。
- 懒绑定 vs hook**：在 `main` 入口断下时，`malloc` 的 GOT 还可能未解析为 `libc` 真实地址，而是 `.plt` 中的 trampoline。如果直接把 GOT 旧值当成 `func_ptr`，调到 trampoline 会进入 `_dl_runtime_resolve` 但找

不到原 GOT 地址而崩溃。因此本实验用 gdb 的 `&malloc` 直接获取 libc 中的真实 malloc 地址，是更鲁棒的做法。

4. `return` `EXPR` 的类型注解：在没有完整调试信息的库（libc、ptrace 等）中执行 `return 0` 会因为推不出返回类型而失败，必须写成 `return (long)0`。